Gym Management System

OOMD Mini Project Report

Object Oriented Modelling and Design

Submitted by _____

Class and Division _____

Roll Numbers _____

Guide _____

Academic Year _____

Java Swing JDBC MySQL

1 Title

Gym Management System

2 Problem Statement

Background

A gym maintains member details, membership types, trainers, attendance, personal-training assignments, fitness measurements, payments, product sales and equipment records. These records refer to the same people and resources but are often kept in separate registers or spreadsheets. Staff must search several places to answer a simple question such as whether a member has attended regularly or which trainer is assigned to that member. Repeated entry also creates spelling mistakes, duplicate records and inconsistent contact details.

Problem to be solved

The project must provide one desktop application that stores related gym information in a MySQL database and presents appropriate actions to each user. The Admin needs to maintain the gym records and user accounts. The Trainer needs to inspect assigned members and record measurements for a current assignment. The Member needs to view personal membership, attendance, progress and training history. These users have different responsibilities, so a successful login must lead to the correct dashboard and enforce access to the permitted records.

Data and access requirements

Member attendance and progress must remain linked to the correct member. A training assignment must identify both its trainer and member. Height, weight and body-fat values must be validated before saving, and BMI must be calculated consistently from height and weight. A trainer must not be able to submit progress for an unrelated or inactive member. The system must check the active assignment again when saving because an assignment may have changed after the form was opened. Inactive accounts or profiles must be rejected at login.

Expected outcome

The application brings administration, trainer work and member self-service into a common local system. Admin can manage member and trainer profiles, memberships, member attendance, equipment, products and account permissions. Search suggestions help locate records. Trainer and Member dashboards provide their permitted histories and limited profile and password updates. JasperReports presents six operational reports, and a local SQL backup action preserves a copy of the database. Payment and sales screens currently support viewing and searching stored transactions; changing a membership payment flag does not create a new payment transaction.

Technology and scope

The mini-project uses Java 17, Swing, JDBC and MySQL with a Windows-oriented local setup. The JVM and a dedicated MySQL instance run on the same computer. Each independent checkout creates its own database files and configuration, using the project scripts and localhost port 3307. The seven UML diagrams explain users, persistent concepts, interactions, workflow, session states, components and deployment. Together they describe the existing implementation and the relationships needed to understand its design.

3.1 Use Case Diagram

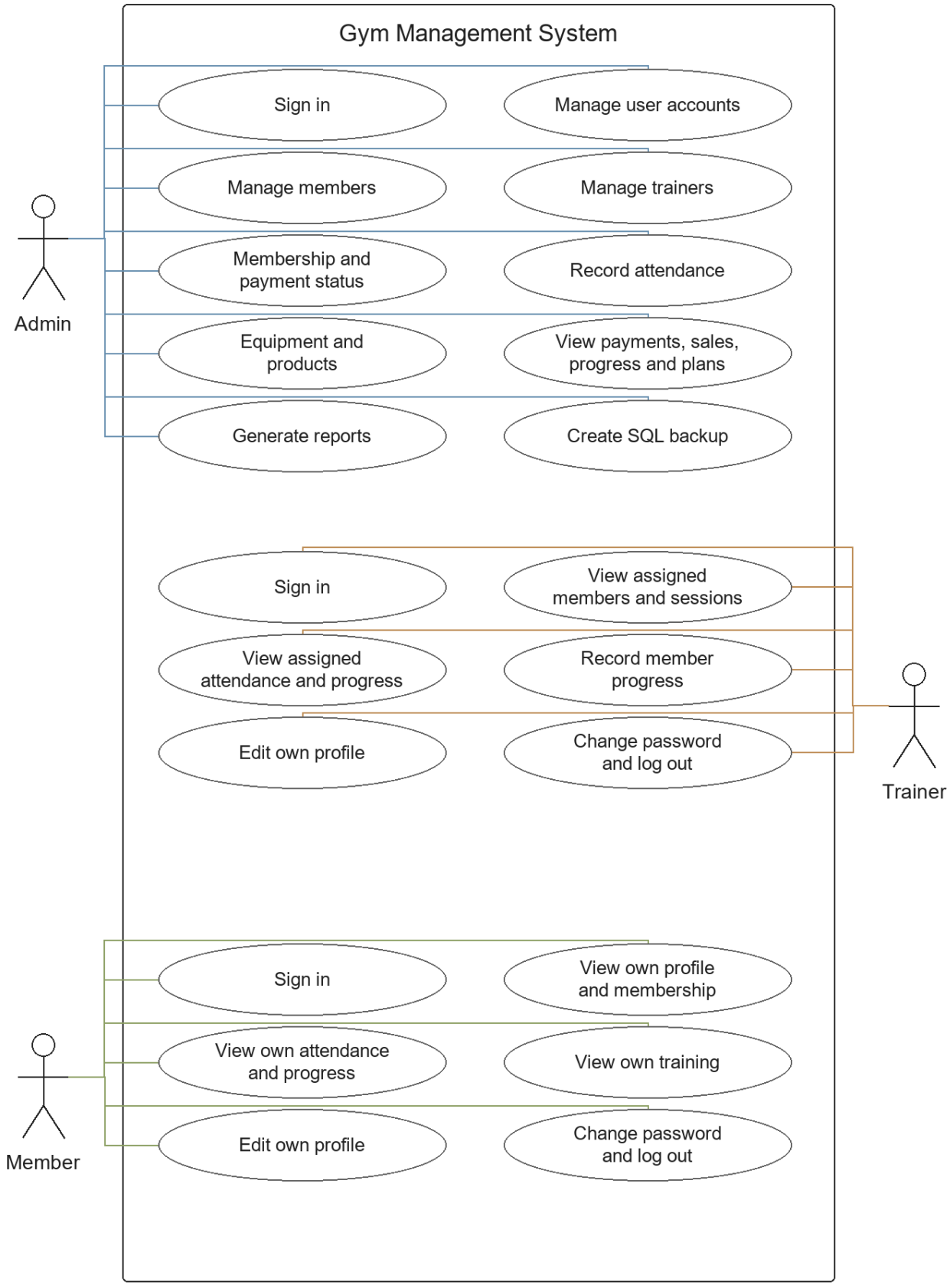


Figure 1 Use Case Diagram

Explanation of the Use Case Diagram

Purpose and notation

The use case diagram shows the Gym Management System from the viewpoint of its users. The outer rectangle represents the application boundary. Each oval describes a task that a user can perform, and the stick figures represent the Admin, Trainer and Member actors. Association lines connect an actor to the tasks available to that role. The layout groups the three roles so their responsibilities can be compared without confusing screens with business tasks.

Admin responsibilities

Admin handles the gym operations. The diagram includes sign in, user account management, member and trainer management, membership and payment status, member attendance, equipment and products. Admin also views payment, sales, progress and workout-plan records, generates reports and creates a SQL backup. User Management controls account roles, activation and linked profiles. The payment and sales goals represent record views: the current application does not provide a complete transaction-entry flow through those screens.

Trainer and Member responsibilities

Trainer can inspect assigned members, personal-training sessions, attendance and progress. Record Member Progress is restricted to an active member with an active assignment to the signed-in trainer. Member can inspect their own profile, membership, attendance, progress and training history. Both roles can edit the allowed personal profile fields, change their password and log out. A Member cannot enter progress measurements or open the administrative inventory and account modules.

Shared login and access rules

All three actors use the same LoginForm. The username or mobile number identifies the stored account, and the account role determines the dashboard. No actor can obtain wider permissions by selecting another role in a menu. Session and account checks enforce access after login as well as at the initial entry point. These checks are represented by the responsibilities shown in the diagram rather than by a separate database actor.

Example and interpretation

For example, a trainer opens Record Progress, selects a currently assigned member and submits the measurements. The same member may later sign in and view that saved row in My Progress. The two users participate in related work, while each receives only the actions appropriate to their role. This diagram establishes who uses the project and what those users expect from it; the later UML diagrams explain how those expectations are implemented.

3.2 Class Diagram

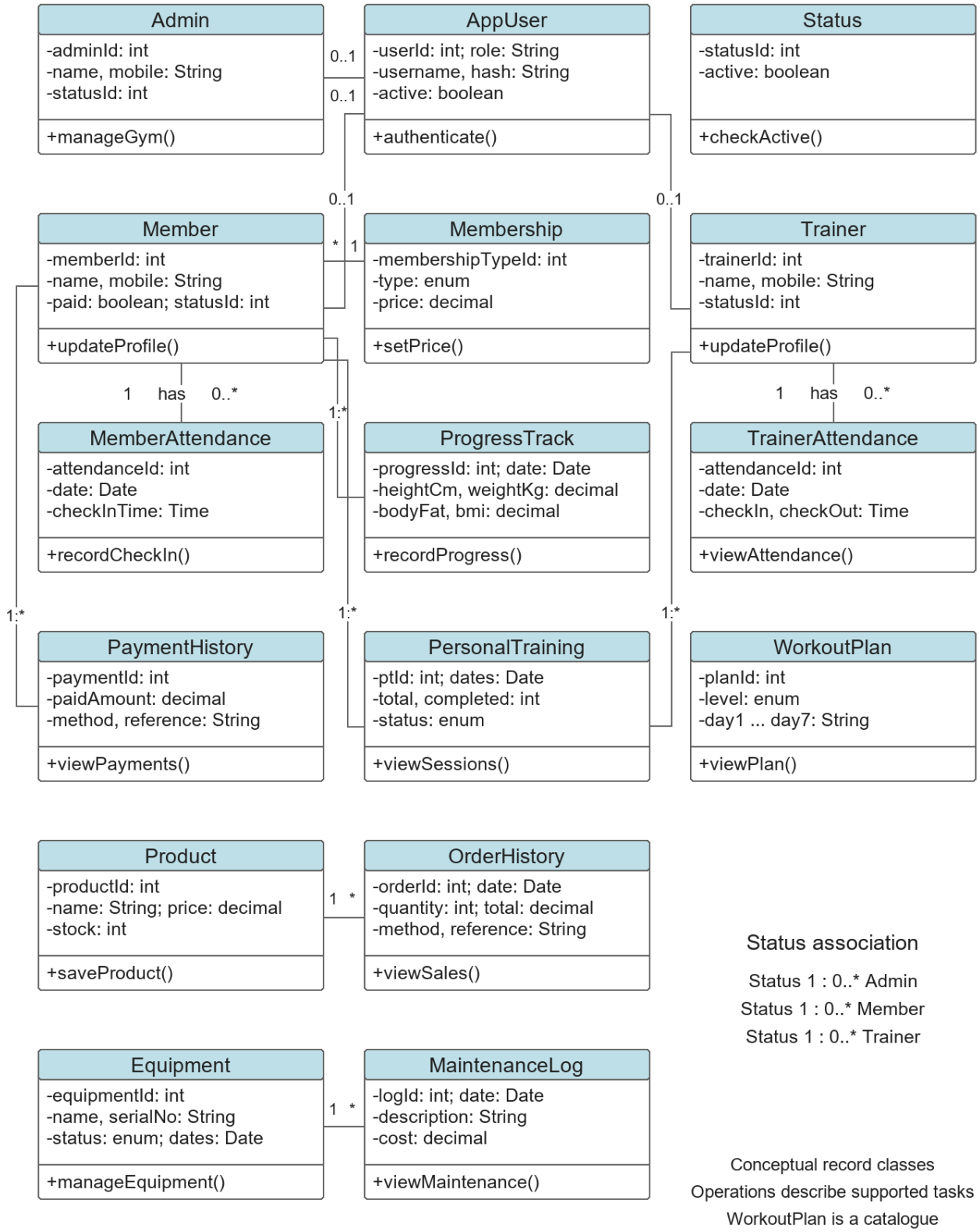


Figure 2 Class Diagram

Explanation of the Class Diagram

Purpose and compartments

The class diagram models the persistent concepts in the gym system. Each class has a light blue name compartment, an attribute compartment and an operation compartment. The minus sign denotes conceptual private attributes and the plus sign denotes conceptual public operations. All sixteen stored record types are represented. The operations describe supported responsibilities; the actual Java project implements them through Swing forms, Accounts, Session and connection.MySQL rather than declaring a Java domain class for every table.

Accounts and profiles

AppUser stores the login account, password hash, role and active flag. An account may have a legacy Admin profile link, or the profile link appropriate to its Trainer or Member role. A profile has at most one linked login account. Member stores personal and payment-status information; Trainer stores trainer details. Both refer to the Status lookup. A Member belongs to one Membership type, and many members can share that type and its price.

History and assignments

MemberAttendance, ProgressTrack and PaymentHistory hold separate dated records for a member. Each row belongs to one member, while a member may have zero or many rows. ProgressTrack stores height, weight, body fat and BMI. TrainerAttendance belongs to a trainer. PersonalTraining links one member with one trainer and records dates, session counts and assignment status. The same trainer and member can appear in several historical assignments. Assignment eligibility for a new progress entry is checked by the account service.

Inventory and supporting records

Product is associated with OrderHistory, where quantity, amount, method and reference information describe a stored sale. Equipment is associated with MaintenanceLog. An item can have several maintenance entries, but each entry identifies one equipment item. WorkoutPlan stores a level and the seven daily plan descriptions. It is a standalone catalogue in the current schema. The diagram therefore does not invent a member-to-plan foreign key. Status is shared by the Admin, Member and Trainer profiles.

Reading the relationships

Solid lines show associations. The multiplicity 1 means one parent, 0..1 means an optional single link, and * means zero or more records. The Status association list names the shared lookup relationships without routing lines across unrelated classes. Associations are used instead of inheritance because Admin, Trainer and Member are role-linked profiles. The class view explains the information model; the component view shows the actual Java modules that perform these operations.

3.3 Sequence Diagram

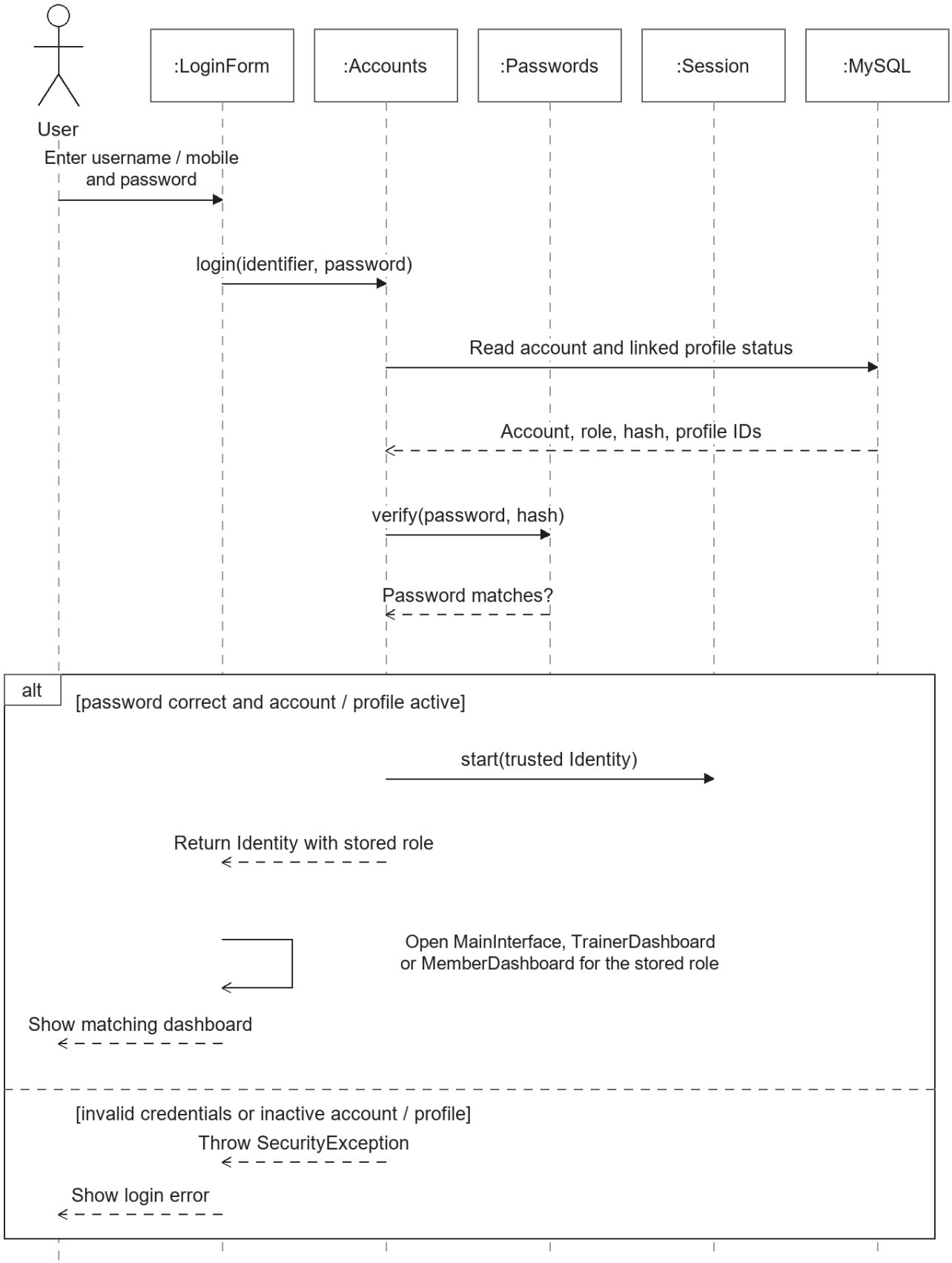


Figure 3 Sequence Diagram

Explanation of the Sequence Diagram

Scenario and notation

The sequence diagram follows one login attempt. The User interacts with LoginForm, and the remaining lifelines represent Accounts, Passwords, Session and MySQL. Time proceeds from the top towards the bottom. Solid message arrows represent calls or input actions; dashed arrows represent responses or errors. The rectangular alt frame contains the successful and unsuccessful outcomes after credential verification.

Account lookup

The user enters a username or mobile number and a password. LoginForm calls Accounts.login with these values. Accounts clears the preceding session and performs the legacy account migration if needed, then reads the account together with the linked profile status from the database. The main diagram concentrates on the normal authentication interaction. MySQL returns the account role, stored hash, profile identifiers and activation values. These values come from the database and form the basis of the access decision.

Password and status verification

Accounts calls Passwords.verify to compare the supplied password with the stored salted PBKDF2 hash. A successful comparison alone is insufficient: the account and any linked profile must also be active. Accounts creates a trusted Identity containing the user ID, username, role and applicable profile ID only after these checks succeed. Session.start stores that identity before the result is returned to LoginForm.

Successful and alternative outcomes

In the successful branch, LoginForm uses the returned role to open MainInterface for Admin, TrainerDashboard for Trainer or MemberDashboard for Member. The user then sees the matching interface. In the alternative branch, invalid credentials or inactive status produce a SecurityException. LoginForm catches it and displays an error. A database failure is also reported by the form. A failed attempt does not establish an authenticated role session.

Design interpretation

The order of messages separates responsibilities: the form collects input, Accounts controls authentication, Passwords handles verification, MySQL supplies stored state and Session holds the accepted identity. Later protected actions call the role and active-account checks again. For example, deactivating an account after login must prevent its next protected operation. This diagram explains the common entry flow used by all three project users without suggesting that the user chooses their own permissions.

3.4 Activity Diagram

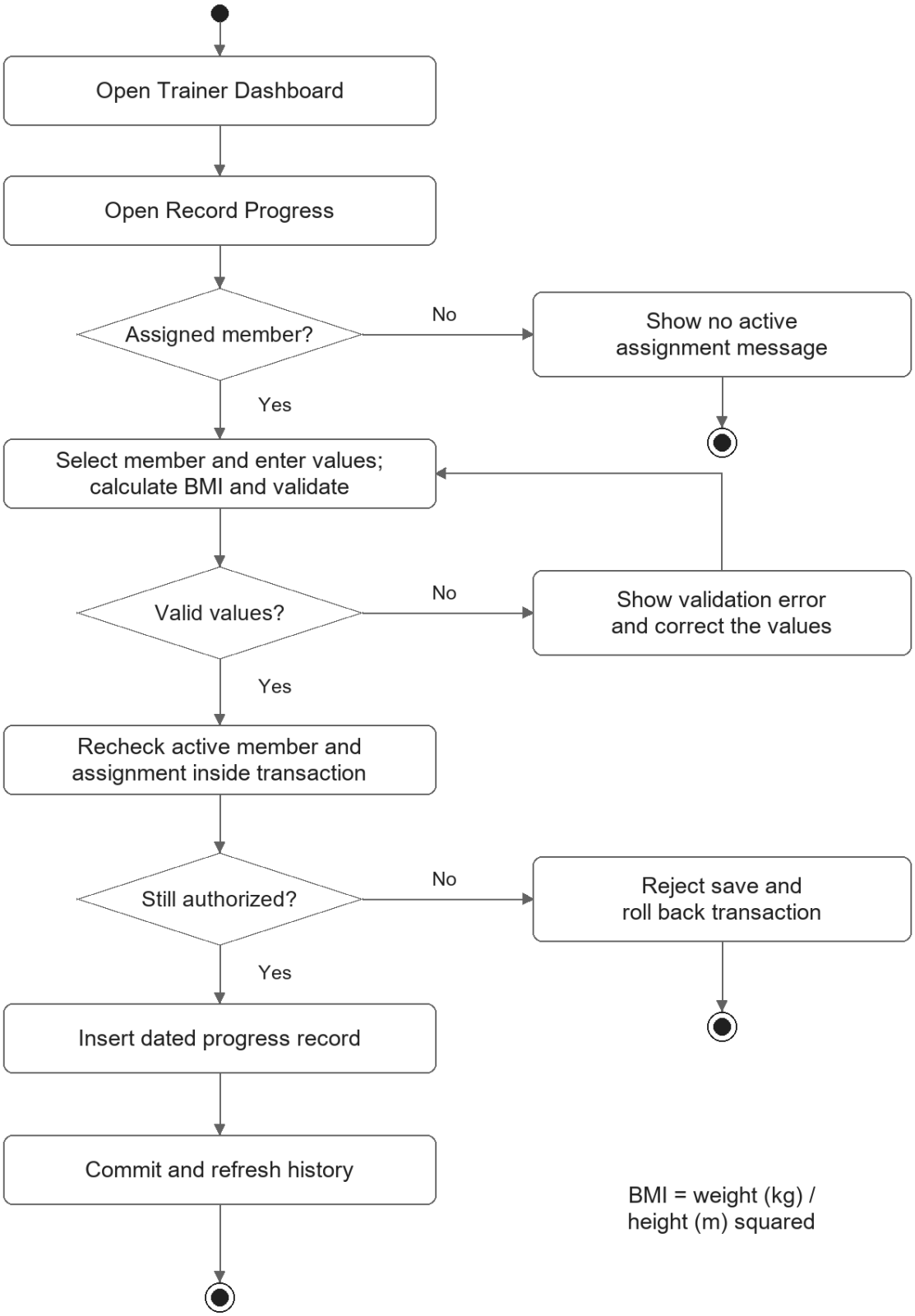


Figure 4 Activity Diagram

Explanation of the Activity Diagram

Workflow and symbols

The activity diagram shows the Trainer workflow for adding a progress measurement. A small filled circle marks the start. Rounded rectangles represent actions, diamonds represent decisions and the bullseye marks completion of a path. The decision branches have Yes and No labels. Actions appear in execution order, and the error paths show when the process ends or returns to an earlier input step.

Member selection

The trainer opens their dashboard and selects Record Progress. The application obtains the eligible members through Accounts.activeMembers. Eligibility requires a personal-training row assigned to the current trainer with ACTIVE status and an active member profile. The signed-in trainer ID comes from Session. If no eligible member is available, the form displays a message and the attempted workflow ends. An eligible member can be selected before measurements are entered.

Input validation

The form collects the measurement date, height in centimetres, weight in kilograms and body-fat percentage. The service rejects a future date or non-finite numeric values. The accepted ranges are height 50 to 250 cm, weight 20 to 400 kg and body fat 1 to 70 percent. Invalid input follows the correction path and does not insert a database record. BMI is calculated as weight divided by the square of height in metres, rounded to two decimal places, and must fit the stored range.

Authorization and save

Before inserting a row, Accounts.recordProgress starts a transaction and rechecks the current member and assignment. This protects against a stale selection if an assignment or profile changed while the form was open. A failed recheck rejects the save and rolls back. When the checks pass, the dated measurements and calculated BMI are inserted into progress_track, the transaction is committed and the dashboard refreshes its displayed progress history.

Interpretation of the paths

The measurement step includes BMI calculation and validation before entering the save transaction. BMI is derived from the supplied weight and height, so the trainer does not type an independent BMI value. Correct values and the current assignment rule must both pass for a committed row. For example, an assignment that becomes inactive after selection follows the rejection path even if every measurement is valid. The workflow therefore shows both data validation and permission to save the record.

3.5 State Machine Diagram

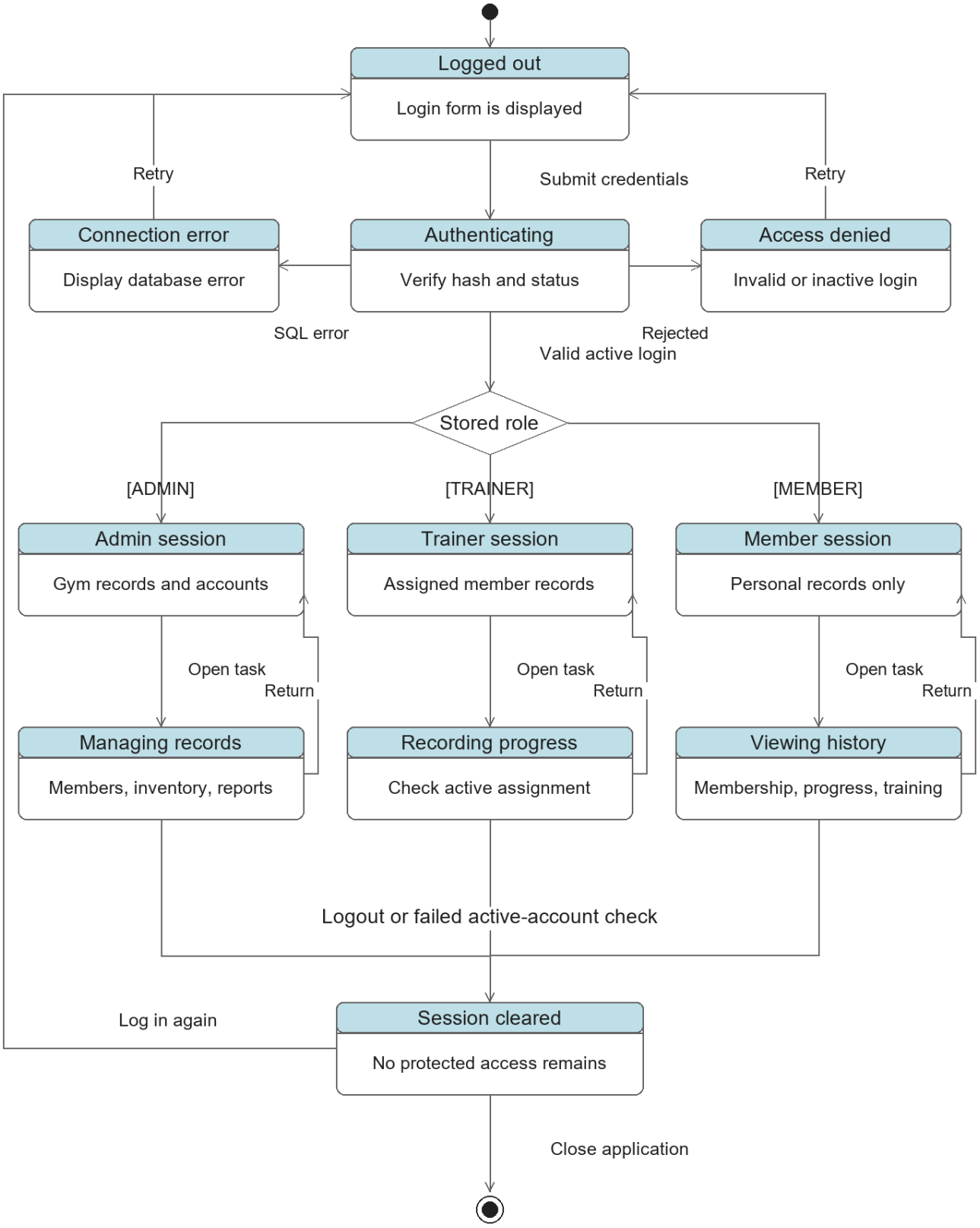


Figure 5 State Machine Diagram

Explanation of the State Machine Diagram

State being modelled

The state machine describes a conceptual lifecycle for the application login session. A state records the condition of the application and the actions available in that condition. The initial marker leads to Logged Out, where LoginForm is displayed. The diagram uses rounded state boxes with a heading and a short description. Transition arrows are labelled with the events or conditions that cause a change.

Authentication and rejection

Submitting credentials enters Authenticating. The application looks up the account, verifies its password hash and checks account and profile activation. A rejected attempt enters Access Denied and displays an invalid or inactive-login message. A database failure enters Connection Error and displays a connection message. Both paths allow another attempt from the login form. These are presentation states derived from the error handling, rather than separate state classes declared in the source.

Role selection and work

A valid login reaches the Stored Role choice. Exactly one of Admin Session, Trainer Session or Member Session is entered according to the database role. Admin works with gym records and accounts. Trainer views assigned records and can record authorized progress. Member views personal membership and history. Opening a task enters the corresponding working state; returning from it preserves the original role. The choice therefore represents alternatives, not three parallel sessions for one login.

Session termination

An explicit logout clears the session. The role dashboards close and the login form becomes available again. A failed active-account check can also clear the identity when the account is deactivated or its role/profile link changes. The diagram uses Session Cleared to show that protected access has ended. Logging in again returns to Logged Out. Closing the application reaches the final marker. Individual windows may handle the resulting error message before this conceptual transition is completed.

Why this view is useful

The state view explains why the same operation may be allowed at one moment and rejected later. A trainer with a valid session still needs an active assignment to save progress, while a member session never gains an administrative state. The model also prevents an incorrect inheritance interpretation of the three roles: they are permitted operating states established by authentication. It complements the sequence diagram, which explains the messages that produce the successful or rejected login transition.

3.6 Component Diagram

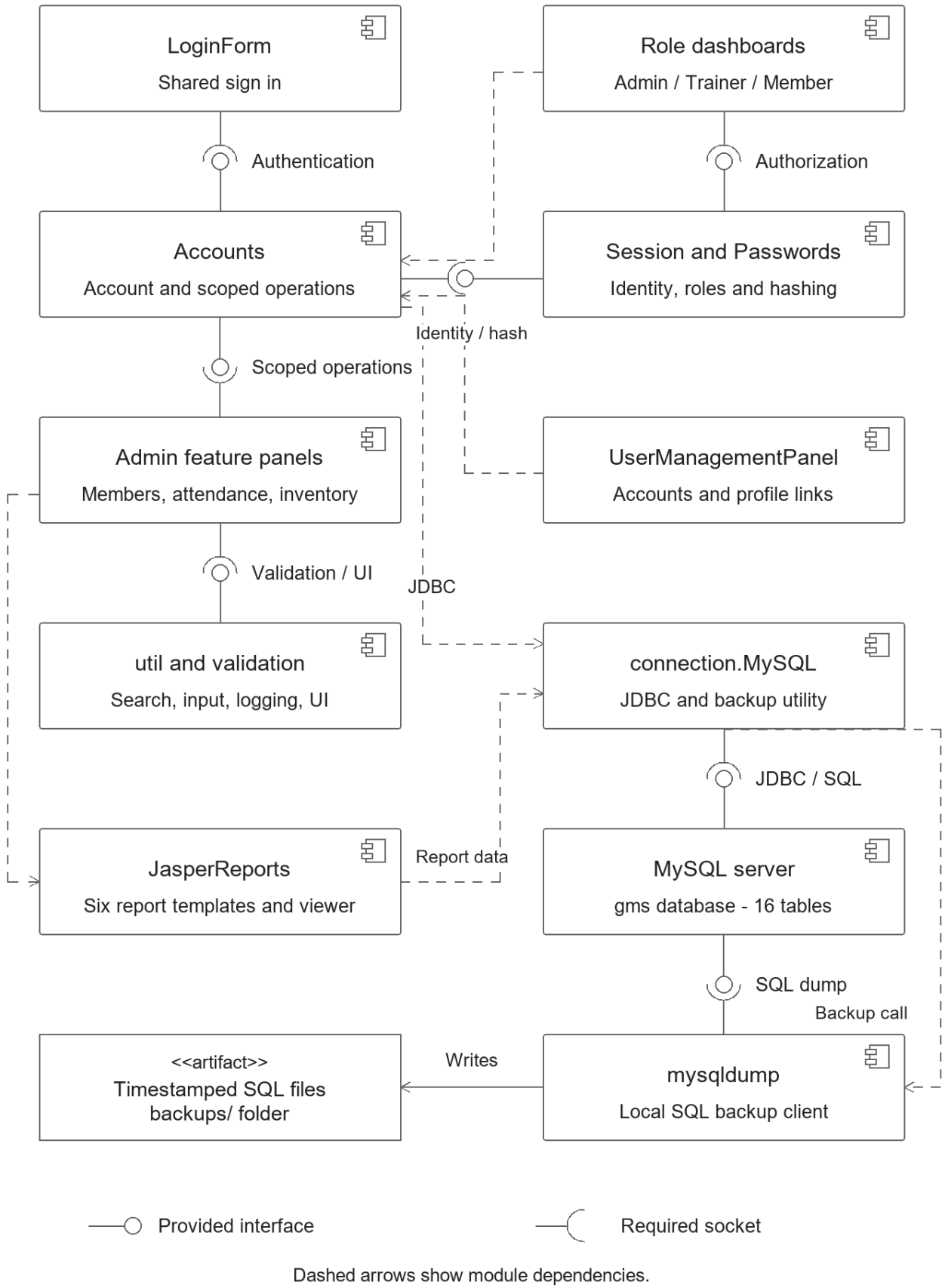


Figure 6 Component Diagram

Explanation of the Component Diagram

Purpose and module notation

The component diagram presents the software modules and the services they require. Each component rectangle carries the small UML component symbol. The boxes identify the common login, role dashboards, account and session helpers, administrative panels, shared utilities, database access and reporting tools. These modules describe the organization of the existing Swing application and its local dependencies.

Provided and required interfaces

A lollipop circle denotes a provided interface. An open semicircle, or socket, denotes a required interface. A connected ball-and-socket pair means one module supplies a service another module needs. LoginForm requires Authentication from Accounts. Role dashboards require Authorization from Session. Admin panels require scoped operations and validation/UI support. Accounts relies on the identity and password services. These labels are conceptual service contracts, not additional Java interface declarations created for this report.

Database and account responsibilities

Accounts encapsulates login, account administration, linked profiles, role-scoped queries and transactional progress entry. Session carries the trusted Identity and checks the current role. Passwords implements hashing and verification. connection.MySQL provides the shared JDBC connection, parameterized SQL helpers for the legacy admin panels and the backup call. MySQL Server supplies the gms schema. The JDBC/SQL assembly identifies the database service needed by the local data utility.

Reports and backup

The Admin report panel loads six compiled JasperReports templates and opens the results in JasperViewer. The report dependency uses the existing database connection to obtain member, trainer, attendance, payment, sales or equipment data. The backup utility invokes mysqldump. That local client requires access to the MySQL SQL-dump service and writes timestamped SQL files in the backups folder. The saved files are shown as an artifact rather than an executable component.

Reading the dependencies

Dashed arrows indicate that the source module depends on the target module. They complement the explicitly drawn interface assemblies and help show dependencies such as UserManagementPanel using Accounts. Shared util and validation classes provide suggestions, input rules, logging and UI helpers. For example, a progress request travels from the Trainer dashboard to Accounts, which checks the session and writes through the JDBC connection. The component diagram identifies who owns each responsibility without treating the application as a web service or a formal MVC framework.

3.7 Deployment Diagram

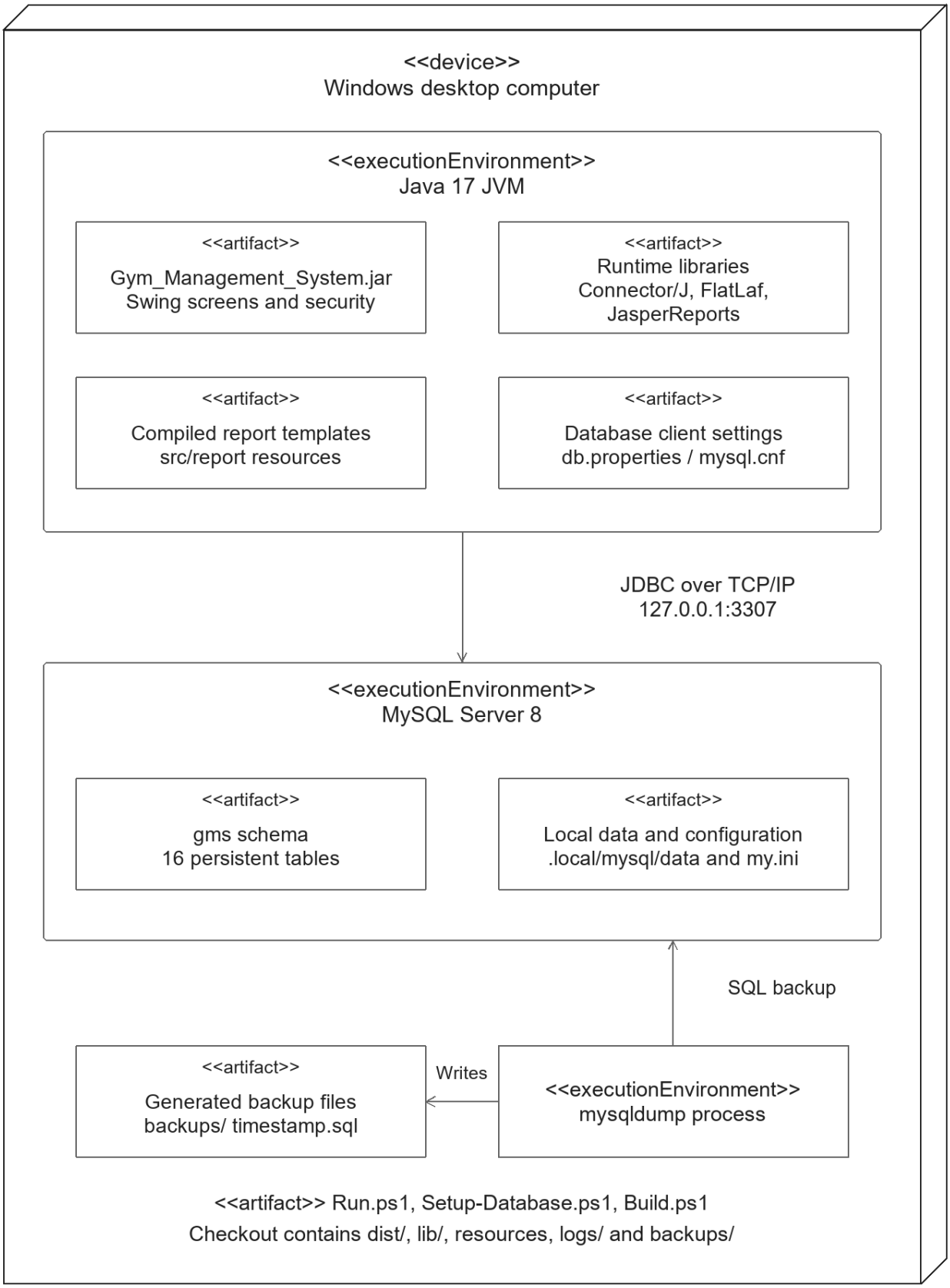


Figure 7 Deployment Diagram

Explanation of the Deployment Diagram

Physical installation and nodes

The deployment diagram places the application on a Windows desktop computer. The three-dimensional outer box is a UML device node. Nested execution-environment boxes represent the Java JVM and the local MySQL Server process. Artifacts are files or packaged resources installed inside this arrangement. The model describes the supplied single-computer setup and the runtime requirements used by this desktop project.

Java execution environment

Java 17 runs Gym_Management_System.jar, which contains the Swing screens and security code. Runtime libraries provide MySQL Connector/J, FlatLaf and JasperReports support. Compiled report templates and UI resources are available to the application. Database client settings are read from the generated properties and MySQL client configuration. The deployment of these resources matters because a copied JAR without its required runtime libraries and configuration may not start correctly.

Database connection and storage

The JVM connects to MySQL using JDBC over TCP/IP at 127.0.0.1:3307. MySQL hosts the gms schema with sixteen tables, including app_user and the member, trainer, attendance, progress and operational records. The checkout-local data directory is .local/mysql/data and the server configuration is .local/mysql/my.ini. The ordinary MySQL service commonly installed on port 3306 is separate. The project setup checks ownership of a listener on 3307 before attempting database work.

Build, launch and backups

Setup-Database.ps1 prepares the dedicated local database, Build.ps1 compiles the project and Run.ps1 performs setup, build and launch. The application uses the local mysqldump executable to create a SQL backup, with the output written under backups. The diagram includes dist, libraries, resources, logs and backup files because the physical installation consists of several resources rather than one executable file. These generated resources stay inside the independent checkout.

Replication requirement

Another computer should use a new checkout, install JDK 17 or newer and MySQL 8 binaries, then follow the supplied setup scripts. The setup must initialize that checkout's own data and configuration; database directories, credentials, generated logs and seed markers are not copied from the original machine. A listener owned by another checkout must not be reused or stopped. This deployment view explains the resources required to reproduce the project and avoids assuming a fixed Windows username or the original Desktop path.